

微服务拓扑故障影响范围分析实验报告

2026 年 5 月 22 日

班级：2024217802 姓名：严子竣 学号：2024212622

1 需求分析

在分布式微服务架构中，服务之间的调用关系可以抽象为有向图，其中顶点代表微服务节点，有向边代表调用关系。当某个服务节点发生故障时，下游无法正常访问该节点的服务数量即为“故障影响范围”。本实验将该工程需求转化为有向图的可达性问题：随机生成符合微服务调用特征的有向图，计算从每个顶点出发所能到达的顶点数量的平均值与最大值，以此评估系统的故障扩散风险，为架构设计和容灾规划提供定量依据。

1.1 程序功能

- 从终端交互接收两个正整数 V （顶点数）和 E （边数），取值范围分别为 $V \geq 1$, $0 \leq E \leq V \times (V - 1)$ 。
- 根据微服务调用特征随机生成简单有向图：前 10% 的顶点视为核心服务，其作为边的源顶点的概率更高（模拟核心服务被更多下游服务调用）。
- 对每个顶点执行广度优先搜索（BFS），计算从该顶点出发可到达的顶点数量（包括自身）。
- 统计所有顶点可达数的平均值和最大值，并依据阈值（平均值/最大值占比 $\geq 50\%$ ）给出故障扩散风险评估结论。

1.2 输入形式与值的范围

- 程序运行后，会提示用户输入两个整数：
 - 顶点数 V ：正整数，取值范围为 $1 \sim$ 任意正整数。
 - 边数 E ：非负整数，最大可能边数为 $V \times (V - 1)$ 。
- 若输入不合法（如 $V \leq 0$ 、 $E < 0$ 或 $E > V(V - 1)$ ），程序会输出错误信息并退出。

1.3 输出形式

- 输出所有顶点可达数的平均值（保留两位小数）、最大值。
- 根据阈值判断系统耦合性：
 - 若平均值占比 $\geq 50\%$ 或最大值占比 $\geq 50\%$ ，输出“说明系统耦合度高”；

- 否则输出“说明系统耦合度低”。
- 若随机生成边时因达到最大尝试次数而未生成全部 E 条边，会输出警告信息。

1.4 测试数据

- 正确输入：例如 $V = 30, E = 100$ ，程序应生成图并输出平均值与最大值。
- 边界输入： $V = 1, E = 0$ （单个节点无边），平均值和最大值均为 1，占比 100
- 错误输入： $V = 0, E = 10$ 或 $V = 5, E = 30$ （超过最大边数 $5 \times 4 = 20$ ），程序提示错误并退出。

2 概要设计

2.1 问题解决思路

程序采用邻接矩阵存储有向图，利用广度优先搜索计算每个顶点的可达节点数。整体流程如下：

1. 接收用户输入的 V 和 E ，进行合法性检查。
2. 初始化 $V \times V$ 的邻接矩阵，所有元素置 0。
3. 定义核心服务顶点集：前 10% 的顶点（索引 0 至 $coreCount - 1$ ），至少包含 1 个顶点。
4. 随机生成 E 条有向边：
 - 每条边以 50% 的概率从核心服务中均匀选取源顶点，以 50% 的概率从普通服务中选取。
 - 目的顶点均匀随机选取。
 - 禁止自环和重复边；若因冲突导致无法生成足够边数，会在达到最大尝试次数后停止并给出警告。
5. 对每个顶点执行 BFS，统计从该顶点出发能到达的顶点数量。
6. 计算所有可达数的平均值与最大值，输出并给出风险评估结论。

2.2 数据结构定义

- 邻接矩阵：使用二维指针 `int **adj`，大小为 $V \times V$ ，`adj[u][v]=1` 表示存在从 u 到 v 的有向边。
- 队列（BFS 用）：

```
1 typedef struct {  
2     int *data;  
3     int front, rear, size, capacity;  
4 } Queue;
```

- 辅助布尔数组：在 BFS 中标记顶点是否已访问。

2.3 主程序流程与模块层次

main

- |— 提示并读取 V, E
- |— 合法性检查
- |— 初始化随机种子
- |— 分配邻接矩阵并清零
- |— 定义核心服务顶点数 `coreCount`
- |— 生成 E 条边（带重试）
 - |— 选择源顶点（核心或普通，概率各半）
 - |— 随机选择目的顶点
 - |— 检查自环和重复边
 - |— 添加边并更新计数
- |— 对每个顶点 i 调用 `bfsReachable`
 - |— 创建访问标记数组
 - |— 创建队列，将 i 入队
 - |— 循环出队，遍历所有邻接顶点
 - |— 统计可达顶点数并返回
- |— 计算平均值、最大值
- |— 输出结果并给出风险评估
- |— 释放邻接矩阵内存

3 详细设计

3.1 核心算法伪代码

3.1.1 随机图生成（含核心服务偏好）

```

1 coreCount = max(V/10, 1)
2 edgesGenerated = 0
3 attempts = 0
4 while edgesGenerated < E and attempts < MAX_ATTEMPTS:
5     if random() % 100 < 50:
6         src = random() % coreCount           // 核心服务
7     else:
8         if V > coreCount:
9             src = coreCount + random() % (V - coreCount) // 普通服务
10        else:
11            continue
12        dst = random() % V
13        if src == dst or adj[src][dst] == 1:
14            attempts++; continue
15        adj[src][dst] = 1
16        edgesGenerated++
17        attempts = 0
18 if edgesGenerated < E:
19     打印警告信息

```

3.1.2 从顶点 s 出发的 BFS 可达计数

```

1 function bfsReachable(adj, V, s) -> int:
2     visited = calloc(V, sizeof(bool))
3     queue = createQueue(V)
4     visited[s] = true
5     enqueue(queue, s)
6     count = 1
7     while not isEmpty(queue):
8         u = dequeue(queue)
9         for v = 0 to V-1:
10             if adj[u][v] == 1 and not visited[v]:
11                 visited[v] = true
12                 count++
13                 enqueue(queue, v)
14     free(visited); freeQueue(queue)
15     return count

```

3.1.3 主控流程

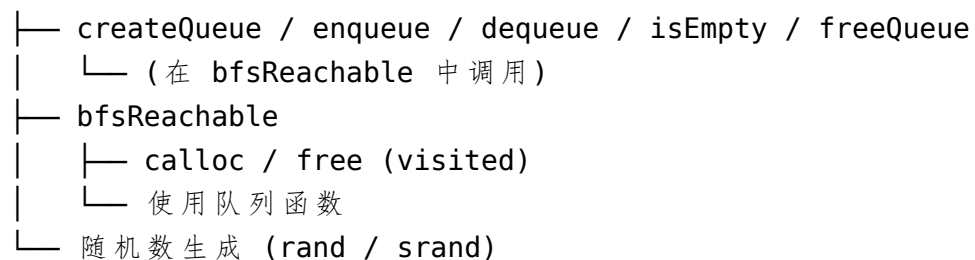
```

1 输入 V, E
2 if V <= 0 或 E < 0 或 E > V*(V-1): 报错退出
3 adj = 分配 V×V 零矩阵
4 coreCount = V/10; if coreCount == 0: coreCount = 1
5 生成边 (如上述)
6 sum = 0; maxReach = 0
7 for i in 0..V-1:
8     reach = bfsReachable(adj, V, i)
9     sum += reach
10    if reach > maxReach: maxReach = reach
11 avg = sum / V
12 输出 avg, maxReach
13 if avg/V >= 0.5 or maxReach/V >= 0.5: 输出 "耦合度高"
14 else: 输出 "耦合度低"
15 释放 adj

```

3.2 函数调用关系图

main



4 调试分析报告

4.1 调试过程中遇到的问题及解决方法

1. 问题 1：重试机制导致死循环或效率低下

当 E 接近最大可能边数 $V(V-1)$ 时，随机选择源和目的很容易遇到已存在的边，导致重试次数急剧增加。

解决方法：设置最大尝试次数 10^6 ，若达到上限则停止添加边并给出警告，同时保留已生成的边。这样可以避免无限循环，且程序仍可继续计算（虽然边数可能少于要求，但不影响演示目的）。

2. 问题 2：核心服务占比小于 1 时的处理

当 $V < 10$ 时， $V/10$ 为 0，但至少需要 1 个核心服务。

解决方法：在计算 `coreCount` 后判断，若为 0 则强制设为 1。

4.2 算法时空复杂度分析

• 时间复杂度：

- 生成边：常数复杂度
- BFS 计算：对每个顶点运行一次 BFS，每次 BFS 遍历所有顶点和边，复杂度 $O(V \cdot (V+E))$ 。对于中等规模图（如 $V \leq 1000$ ）可接受。

• 空间复杂度：

- 邻接矩阵： V^2 个整数，约为 $4V^2$ 字节。
- BFS 辅助数组： $O(V)$ 。
- 队列： $O(V)$ 。
- 总空间复杂度 $O(V^2)$ ，当 V 较大时内存占用显著。

4.3 改进设想

- 对于大规模图，邻接矩阵内存过大，可以在边较少时改用邻接表存储。
- 增加对生成图的统计信息输出，如平均出度、入度分布等。
- 支持从文件读取边列表，而非随机生成，以分析真实拓扑。

5 用户使用说明

1. 编译程序：gcc -o graph_exp chap7.exp.c -O2

2. 运行程序：./graph_exp

3. 根据提示输入顶点数 V 和边数 E ，例如：

请输入顶点数 V （正整数，取值范围：1 ~ 任意正整数）：30

请输入边数 E （非负整数，最大可能边数为 $V \cdot (V-1)$ ）：100

4. 程序输出平均值和最大值，并给出耦合性结论，例如：

平均值为 16.23，最大值为 28，说明系统耦合度高

5. 若边数超过最大可能值，程序会提示错误并退出。

6 测试结果

6.1 测试环境

- 操作系统：Windows 11
- 编译器：MinGW-W64 GCC 8.1.0 / GCC 9.4.0
- 编译选项：-std=c11 -Wall

6.2 测试用例 1：中等规模图（V=30, E=100）

测试步骤：运行程序，输入 30 和 100。

结果：平均值为 26.13，最大值为 30，说明系统耦合度高

6.3 测试用例 2：稀疏图（V=50, E=30）

测试数据：50 个顶点，30 条边。

结果：平均值为 1.78，最大值为 14，说明系统耦合度低

6.4 测试用例 3：完全图（V=10, E=90）

测试数据：10 个顶点，最大边数为 90，E=90 表示完全有向图（无自环）。

结果：平均值为 10.00，最大值为 10，说明系统耦合度低

6.5 测试用例 4：单顶点（V=1, E=0）

测试数据：1 个顶点，0 条边。

结果：平均值为 1.00，最大值为 1，说明系统耦合度高

6.6 测试用例 5：非法输入（V=0, E=5）

测试数据：顶点数 0。

结果：程序输出“错误: V 必须为正整数（1）。”并退出。符合预期。

6.7 测试用例 6：边数超限（V=5, E=30）

测试数据：5 个顶点，最大边数为 20，输入 30。

结果：程序输出“错误: E 不能超过 $V*(V-1)=20$ 。”并退出。错误检测有效。